

BACKEND INTERNSHIP - WEEK 1

Welcome

Welcome to the AllyGo Backend Team.

About AllyGo:

AllyGo is a task marketplace that connects users who need help with everyday tasks to trusted helpers.

The platform includes:

- User Authentication
- Task Marketplace
- Escrow Payments
- QR Verification
- Notifications
- Reviews & Ratings
- Admin Dashboard

Before writing a single line of code, please understand how our development workflow works.

Project Repository

The complete backend source code is available on our GitHub repository.

You will be provided access to:

- Backend Repository
- Development Branch
- Project Documentation
- Postman Collection
- Database Schema
- Railway Development Environment

Before You Start

Before writing any code:

- Read the README
- Understand the folder structure
- Run the backend locally
- Review the project architecture
- Understand existing APIs
- Ask questions if anything is unclear

Do not start implementing features without understanding the existing architecture.

Development Workflow

We follow an industry-standard Git workflow.

1. Never push directly to main

The main branch is protected.

All work must go through Pull Requests.

2. Create your own branch

Naming convention:

```
feature/your-name/task-name
```

Example

```
feature/rahul/notification-service  
feature/priya/user-search  
feature/anand/payment-webhook
```

3. Pull latest changes

```
git checkout develop  
git pull origin develop
```

4. Create feature branch

```
git checkout -b feature/your-name/task-name
```

5. Commit frequently

Good

```
git commit -m "Add task search endpoint"  
git commit -m "Fix notification bug"  
git commit -m "Improve validation"
```

Avoid

```
git commit -m "Done"  
git commit -m "Changes"  
git commit -m "Update"
```

6. Push your branch

```
git push origin feature/your-name/task-name
```

7. Create Pull Request

Create a PR against

`develop`

NOT

`main`

8. PR Description

Every PR should contain

- Summary
- What was implemented?
- Files Changed
- Testing Done
- Screenshots (if required)
- Known Issues
- Checklist

Development Rules

- Never push directly to main
- Never merge your own PR
- Keep PRs small
- One feature per branch
- Do not modify unrelated files
- Follow existing code style
- Use meaningful commit messages

Proposed Tech Stack

LAYER	TECHNOLOGY	PURPOSE / NOTES
Backend	Node.js + Express.js + TypeScript	REST API development with a scalable and type-safe architecture.
Database	PostgreSQL (Neon or Supabase)	Primary relational database for application data.
ORM	Prisma ORM	Type-safe database access, schema management, and migrations.
Cache & Queue	Redis (Upstash) + BullMQ	Caching, background jobs, scheduled tasks, and queue processing.
Mobile Application	React Native + Expo (Managed Workflow)	Cross-platform mobile application for Android and iOS.
Admin Panel	Next.js (App Router)	Internal dashboard for user, task, payment, and analytics management.
Frontend Hosting	Vercel	Hosting and deployment for the admin panel.
Payments & Escrow	Razorpay Route	Payment processing, escrow handling, payouts, and refunds.
Authentication		

LAYER	TECHNOLOGY	PURPOSE / NOTES
	Phone OTP (MSG91 or Firebase Auth) + JWT + Refresh Tokens	Secure user authentication and session management.
File Storage	Cloudflare R2	Storage for uploaded documents, certificates, profile images, and task files.
Backend Hosting	Railway or Render	Cloud hosting for the backend API and services.
Monitoring & Error Tracking	Sentry	Real-time error monitoring, performance tracking, and crash reporting.
Product Analytics	PostHog	User behavior analytics, feature adoption, and event tracking.
Push Notifications	Expo Push Notifications	Send real-time notifications to mobile users.
Version Control	Git + GitHub	Source code management, collaboration, and Pull Request workflow.
API Style	REST API	Standardized communication between frontend, mobile, and backend services.
Package Manager	npm or pnpm	Dependency and package management.
Environment Management	dotenv	Secure management of environment variables and configuration.

Folder Structure:

Folder Structure

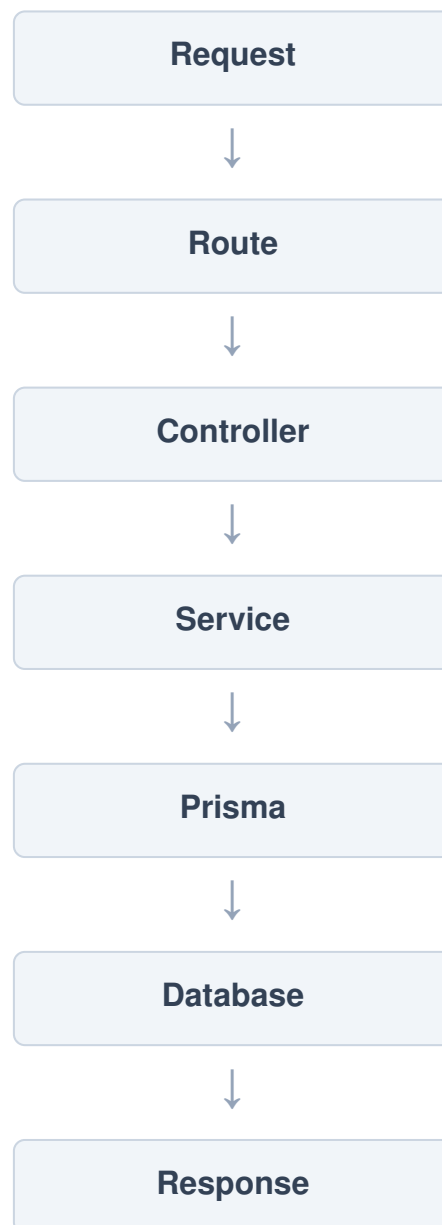
Before implementing any feature, review:

- Folder Structure
- Modules
- Controllers
- Services
- Routes
- Middleware
- Utilities

- Prisma Schema
- Validation Schemas

Understanding the existing project structure is mandatory before development.

Project Architecture:



Week 1 Learning Objectives

By the end of Week 1, interns should understand:

- Project Architecture

- Express Backend
- TypeScript
- Prisma ORM
- Database Design
- Authentication Flow
- Error Handling
- Git Workflow
- Pull Requests
- API Development

Task 1 – Project Setup

Objective

Deliverables

- Clone repository
- Install dependencies
- Configure .env
- Generate Prisma Client
- Run Prisma migrations
- Seed database
- Run backend successfully
- Test APIs using Postman

Task 2 – Understand Database and Api

Study

Understand

- Purpose
- Relationships
- Foreign Keys
- Lifecycle
- Which APIs use each model

Document

- Endpoint
- Controller
- Service
- Validation
- Response
- Authentication

Task 3: Implement a robust and scalable User Search API that allows users to search for other users within the AllyGo platform. The API should support searching by name and username while providing pagination, sorting, and proper validation.

Feature Requirements

Endpoint

GET /api/v1/users/search

Authentication

JWT Authentication is required.
Only authenticated users can access this endpoint.

Query Parameters

PARAMETER	TYPE	REQUIRED	DESCRIPTION
q	string	Yes	Search keyword (minimum 2 characters)
page	number	No	Default: 1
limit	number	No	Default: 10
sortBy	string	No	name, username, createdAt
order	string	No	asc / desc

Example:

```
GET /api/v1/users/search?q=john&page=1&limit=10
```

Search Behaviour

The API should support:

- Search by full name
- Search by username
- Partial matching
- Case-insensitive search
- Ignore leading/trailing spaces
- Return an empty array if no users are found

Response

Example Success Response

```
{
  "success": true,
  "data": [
    {
      "id": "...",
      "fullName": "John Doe",
      "username": "johndoe",
      "profileImage": "...",
      "rating": 4.8
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 57,
    "totalPages": 6
  }
}
```

Validation

Validate all incoming query parameters using Zod.

Handle cases such as:

- Missing search query

- Search query shorter than 2 characters
- Invalid page
- Invalid limit
- Invalid sortBy
- Invalid order

Return standardized validation errors.

Database

Use Prisma ORM.

The implementation should be optimized and avoid unnecessary database queries.

Exclude

The API should not return:

- Current logged-in user
- Soft-deleted users (if applicable)
- Blocked/Banned users (if applicable)

Sorting

Support sorting by:

- Full Name
- Username
- Created Date

Both ascending and descending order.

Pagination

Implement server-side pagination.

Response should include:

- Current Page
- Total Pages
- Total Records
- Records Per Page

Code Structure

Follow the existing project architecture.

Implement changes in the appropriate:

- Route

- Controller
- Service
- Validation Schema
- Prisma Query Layer

Do not place business logic inside controllers.

Error Handling

Use the centralized error handling system.

Avoid:

- `console.log()`
- Unhandled exceptions
- Generic error messages

Testing

Verify the API for:

- Successful search
- Empty search result
- Invalid query parameters
- Pagination
- Sorting
- Authentication failure
- Large datasets

Deliverables

- Complete implementation
- Proper validation
- Clean, readable, and maintainable code
- Updated API documentation
- Postman collection with sample requests
- GitHub Pull Request

Pull Request Checklist

Before Opening a PR

☐ Code builds

☐ No TS Errors

☐ API Tested

☐ Documentation Updated

☐ No Console Logs

☐ No Dead Code

☐ Feature Works

Communication:

If you're blocked for more than 30 minutes,
ask for help.

Don't spend hours debugging silently.

Share daily progress.

Inform mentor before changing architecture.